

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V College of Science and
Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Dot.Net
Practical II

Name:-Sagar Kandari	Roll no:-T010
Batch:-1	Class:-TYIT
Date of practical:-9/7/26	Date of submission:-9/7/26

PART – A

Write the program with different features of C#

a)Function Overloading

Code:

```
/*using System;
```

```
class A
```

```
{  
    public void displayA()  
    {  
        Console.WriteLine("Display A Function present in class A");  
    }  
}
```

```
class B : A
```

```
{  
    public void displayB()  
    {  
        Console.WriteLine("Display B Function present in class B");  
    }  
}
```

```
class C : A
```

```
{  
    public void displayC()  
    {  
        Console.WriteLine("Display C Function Present in class C");  
    }  
}
```

```
class MainClass
```

```
{  
    public static void Main(string[] args)  
    {  
        B b = new B();  
    }  
}
```

```

        C c = new C();

        b.displayA();
        b.displayB();

        c.displayA();
        c.displayC();

        Console.WriteLine("Sagar");
    }
}*/
using System;

class Demo
{
    void Show(int a)
    {
        Console.WriteLine("Number: " + a);
    }

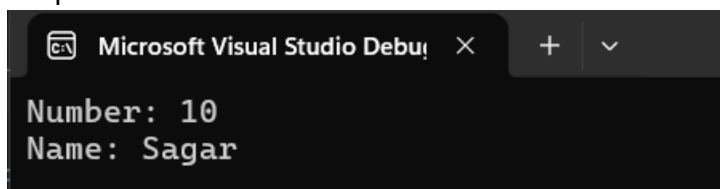
    void Show(string s)
    {
        Console.WriteLine("Name: " + s);
    }

    static void Main()
    {
        Demo obj = new Demo();

        obj.Show(10);
        obj.Show("Sagar");
    }
}

```

Output:



The screenshot shows the Microsoft Visual Studio Debug Console window. The title bar reads "Microsoft Visual Studio Debug Console". The output text is as follows:

```

Number: 10
Name: Sagar

```

b) Inheritance (all types)

Code single level:

using System;

```
class A
{
    public void displayA()
    {
        Console.WriteLine("Display A Function present in class A");
    }
}

class B : A
{
    public void displayB()
    {
        Console.WriteLine("Display B Function present in class B");
    }
}

class C : B
{
    public void displayC()
    {
        Console.WriteLine("Display C Function Present in class C");
    }
}

public class main
{
    public static void Main(string[] args)
    {
        C c = new C();
        c.displayA();
        c.displayB();
        c.displayC();
        Console.WriteLine("Sagar");
    }
}
```

Output:

Output:

```
Display A Function present in class A
Display B Function present in class B
Display C Function Present in class C
Sagar
```

c) Constructor overloading

Code:

Output:

d) Interfaces

Code:

using System;

```
public interface A
{
    void displayA();
}
```

```
public interface B
{
    void displayB();
}
```

```
class C : A, B
{
    public void displayA()
    {
        Console.WriteLine("Display A Function present in interface A");
    }

    public void displayB()
    {
        Console.WriteLine("Display B Function present in interface B");
    }

    public void displayC()
    {
        Console.WriteLine("Display C Function Present in class C");
        Console.WriteLine("Sagar");
    }
}
```

```
class MainClass
{
    public static void Main(string[] args)
    {
        C c = new C();

        c.displayA();
        c.displayB();
    }
}
```

```

        c.displayC();
    }
}

```

Output:

```

Display A Function present in interface A
Display B Function present in interface B
Display C Function Present in class C
Sagar

```

e) Using Delegates and events

Code:

```

using System;
public delegate void Notify();
public class Process
{
    public event Notify Completed;
    public void Start()
    {
        Console.WriteLine("Process started");
        OnCompleted();
    }
    protected virtual void OnCompleted()
    {
        Completed?.Invoke();
    }
}
public class MainClass
{
    static void Done()
    {
        Console.WriteLine("Event triggered: Process completed");
    }

    public static void Main(string[] args)
    {
        Process process = new Process();
        process.Completed += Done;
        process.Start();
        Console.WriteLine("Sagar");
    }
}

```

Output:

```
Output:
Process started
Event triggered: Process completed
Sagar
```

(f) Exception handling

Code:

using System;

public class MainClass

```
{
    public static void Main(string[] args)
    {
        try
        {
            Console.WriteLine("sagar");
            int a = 10;
            int b = 0;
            int result = a / b;
            Console.WriteLine("Result: " + result);
        }
        catch (DivideByZeroException)
        {
            Console.WriteLine("Exception caught: Cannot divide by zero");
        }
        finally
        {
            Console.WriteLine("Program completed");
        }

        Console.WriteLine("Sagar");
    }
}
```

Output:

```
Output:
sagar
Exception caught: Cannot divide by zero
Program completed
Sagar
```